

ControlPlane.ai

Product Requirements & Technical Specification Document

Prepared for: Accenture Innovation Challenge 2026 | Team: 23je0841

1. Executive Summary

Current enterprise AI deployments suffer from a critical operational blind spot: **After the Fact Discovery** [cite: 1]. Organizations deploy powerful models but lack real-time telemetry across inference planes, often relying on downstream user feedback to catch outputs that are confidently wrong, prohibitively expensive, or dangerously biased [cite: 1, 2].

ControlPlane.ai resolves this by introducing a non-blocking sidecar proxy that intercepts LLM payloads at the network layer [cite: 1]. By decoupling generation from evaluation through an asynchronous architecture [cite: 1], it provides live, multi-dimensional oversight (Performance, Cost, Responsibility) without introducing latency to the primary user experience [cite: 1, 2]. It transforms AI oversight from reactive damage control to proactive, real-time mitigation, catching errors before they reach the user [cite: 2].

2. Product Requirements Document (PRD)

2.1 Product Vision & Goals

- **Vision:** To become the ubiquitous trust and telemetry layer for enterprise AI, ensuring every token generated is factually grounded, cost-efficient, and enterprise-compliant before it reaches the end user [cite: 1, 2].
- **Primary Goal:** Build a scalable interception layer that evaluates LLM outputs across three core vectors—Hallucinations (Performance), Compute Burn (Cost), and Toxicity/Data Leakage (Responsibility) [cite: 1, 2].

2.2 Core Features & User Stories

Feature 1: Non-Blocking Sidecar Interception

Requirement: The evaluation layer must operate asynchronously from the primary data stream, ensuring user-facing applications experience zero perceived latency during the oversight process.

- Must intercept TCP/WebSocket connections silently [cite: 1].
- Must fork the payload stream to the evaluation engine while allowing the primary stream to pass through immediately to the user [cite: 1].

Feature 2: Multi-Dimensional Micro-Agent Evaluation

Requirement: Deploy specialized micro-agents for parallel evaluation across distinct risk categories [cite: 1].

- **Performance:** Detect hallucinations against enterprise data and assess system confidence [cite: 1, 2].
- **Cost:** Track token usage and compute trajectory dynamically to prevent excessive billing [cite: 1, 2].
- **Responsibility:** Scan for PII, toxic semantics, and data leakage [cite: 1, 2].

Feature 3: Dynamic Action Engine

Requirement: The system must automatically mitigate risks based on severity, deciding instantly whether to pass, edit, or block the output [cite: 1, 2].

3. Technical Specification Document

3.1 System Architecture Overview

ControlPlane.ai utilizes an asynchronous, event-driven architecture designed to operate at the network edge [cite: 1]. The core paradigm relies on intercepting the LLM stream, immediately forwarding it to the client buffer, and simultaneously feeding it into a high-throughput message broker (Kafka/Redis) for parallel processing by specialized analytical micro-agents [cite: 1].

To orchestrate these parallel micro-agents effectively, the backend leverages scalable Python-based agentic workflows. This robust orchestration layer manages the state and routing of evaluation tasks, ensuring that the intensive processes of semantic similarity checks and toxicity classification do not bottleneck the network proxy [cite: 1].

3.2 Component Specifications

3.2.1 Network Proxy Layer (Sidecar)

- **Function:** Sits alongside the application container. Intercepts outbound requests to the LLM provider and inbound streaming responses without blocking the network layer [cite: 1].

3.2.2 Asynchronous Event Bus

- **Technology:** Redis / Apache Kafka [cite: 1].
- **Function:** Decouples the proxy from the evaluation layer. Handles high-velocity pub/sub messaging [cite: 1]. The proxy publishes the intercepted payload; the micro-agents subscribe and consume it.

3.2.3 Evaluation Micro-Agents

These agents run continuously in the background, decoupled from the generation process to eliminate system bottlenecks [cite: 1]:

- **Performance Agent (Anti-Hallucination):** Utilizes optimized RAG pipelines for verification [cite: 1]. It employs quantized Cross-Encoders to compute real-time semantic similarity scores between the LLM output and grounded enterprise context vectors [cite: 1].
- **Cost Telemetry Agent:** Analyzes active generation loops using streaming token heuristics [cite: 1]. It leverages Redis caching and applies dynamic rate-limiting to prevent compute burn [cite: 1].
- **Responsibility Agent (Guardrails):** Deploys ultra-fast Named Entity Recognition (NER) pipelines and lightweight toxicity classifiers to scan outbound data, preventing data leakage and enforcing enterprise guardrails [cite: 1].

3.3 Dynamic Action Engine & Mitigation Protocol

The system categorizes real-time telemetry into three mitigation tiers, acting on flagged telemetry without crippling the primary LLM pipeline [cite: 1]:

Risk Level	Scenario	Action Protocol	Latency Impact
LOW RISK	Pass-Through	Stream proceeds uninterrupted. Asynchronous background evaluation confirms safety. [cite: 1]	< 50ms TTFT overhead [cite: 1]

Risk Level	Scenario	Action Protocol	Latency Impact
MEDIUM RISK	Auto-Edit	If PII is detected mid-stream, a lightweight Regex/NER mask applies inline replacement before flushing the buffer to the client. [cite: 1]	Minimal (Inline masking) [cite: 1]
HIGH RISK	Block & Escalate	For severe toxicity or massive context hallucination, the proxy forcefully tears down the TCP/WebSocket connection and fires a webhook to the HITL dashboard. [cite: 1]	Connection Terminated [cite: 1]